

Medical Report Timeline Analyzer

- Ibrahim Butt (bscs24043)
- Jaffar Kazmi (bscs24139)

Introduction

Medical Report Timeline Analyzer helps users understand laboratory reports without medical training. You can:

- Upload medical reports as **PDF** or **image** (JPG, PNG, etc.)
- Extract test names and values using **OCR** and **AI**
- Read a **plain-English summary** of your latest report
- **Ask questions** about the uploaded report through a constrained AI chat
- View **health trends** over time on interactive charts
- Browse a full **history** of previously analyzed reports

Important: This application is an educational tool. It does **not** diagnose conditions, prescribe medication, or replace a qualified healthcare professional. Always consult your doctor for medical decisions.

Tech Stack

- **Frontend:** Next.js (React), TailwindCSS, Recharts, Web Speech API
- **Backend:** Python, FastAPI, Uvicorn, Pydantic
- **AI & Parsing:** Groq API (LLaMA 3 Vision & 70B text models), PyMuPDF
- **Database & Auth:** Supabase (PostgreSQL)

Software Engineering Architecture & Principles

To ensure maintainability and scalability, MedAnalyzer strictly adheres to industry-standard software engineering practices.

1. Architectural Style

- **Layered Architecture:** The system is strictly divided into decoupled layers to enforce separation of concerns:
 - **Presentation Layer (UI):** Built with Next.js, handling state, charts, and user interaction.
 - **Business Logic Layer:** Built with FastAPI, handling prompt construction, data validation, and AI orchestration.
 - **Data Access Layer:** Supabase client managing PostgreSQL reads and writes.

2. Paradigms & Design Patterns

- **Object-Oriented Paradigm:** Utilized heavily in the backend via Pydantic (`class ChatRequest(BaseModel)`), encapsulating data structures and enforcing strict typing for API payloads.
- **Facade Pattern:** The FastAPI backend acts as a facade. The Next.js frontend is completely shielded from the complex mechanics of PyMuPDF parsing and Groq LLM multimodal requests. It simply interacts with two clean endpoints (`/api/analyze` and `/api/chat`).
- **Adapter Pattern:** The backend `chat_with_report` endpoint acts as an adapter. It takes the simple chat history array generated by the React frontend and translates it into the strict `[{"role": "system", ...}, {"role": "user", ...}]` JSON schema required by Groq's LLaMA models.

3. Development Methodology

- **Agile Manifesto:** The project prioritized working software and iterative improvements. Features like Text-to-Speech and chat memory were added incrementally over a functional base.
- **Kanban:** Development followed a task-pulling workflow. Individual features (e.g., "Fix Docker build", "Implement Urdu TTS") were isolated and completed before moving to the next.
- **Code Review:** Collaborative development required reviewing partner commits, managing Git stash/pop operations, and resolving merge conflicts
- **Testing:** Continuous integration testing was performed using the FastAPI Swagger UI (`/docs`) and browser Network tabs to verify data flow across Cloudflare tunnels.

System Requirements

For end users (deployed version)

- A web browser
- Internet connection
- A valid email address to create an account
- A medical report file in **PDF** or **image** format

Installation Guide

Clone the project repo

```
git clone https://github.com/ibmbt/report_analyzer.git
cd report_analyzer
```

Backend setup

Backend Dependencies

```
cd backend
cp .env.example .env
pip install fastapi uvicorn python-multipart python-dotenv pypdf pillow pydantic
supabase groq
```

fill the .env file

Start the API server:

```
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Verify it is running by opening: <http://localhost:8000>

You should see: {"status": "Processing Layer Active"}

Supabase database setup

go to supabase website, create account and project, navigate to SQL Editor and run the following..

```
CREATE TABLE reports (
  id SERIAL PRIMARY KEY,
  user_id UUID NOT NULL, -- This will tie into Supabase Auth later
  report_date TIMESTAMPTZ DEFAULT NOW(),
  extracted_metrics JSONB NOT NULL, -- Perfect for dynamic medical key-value
pairs
  ai_summary TEXT,
  raw_ocr_text TEXT
);
```

Frontend setup

on a new terminal:

```
cd frontend
cp .env.example .env.local
npm install
```

fill the .env.local file

Start the web app:

```
npm run dev
```

Open the application in your browser: <http://localhost:3000>

Getting Started

1. Open the application URL in your browser.
 2. On the login screen, enter your email and password.
 3. Click Sign Up to create a new account, or Sign In if you already have one.
 4. After login, you are taken to the Dashboard with the sidebar: Dashboard, Timeline, and History.
-

Feature Walkthrough

Upload and analyze a medical report

Navigation: Dashboard → Overview

1. Under Analyze New Report, click the upload area (Click to select Image or PDF).
2. Choose a lab report file (.pdf, .jpg, .png, or similar).
3. Confirm the file name appears below the upload box.
4. Click Extract Data.
5. Wait while the system:
 - Extracts text (PDF parsing or OCR for images)
 - Uses AI to identify test names and values
 - Generates a short summary
 - Saves the result to your account history

What you see after processing:

- Latest Analysis - AI summary in simple English
- Metric cards - each extracted test name and value

If upload fails:

- Ensure the backend is running (NEXT_PUBLIC_API_URL must be correct).
 - Use a clear, readable scan or photo for image files.
 - Supported formats: images and PDF only.
-

Ask AI about your report

Navigation: Dashboard → Overview (after a successful upload)

1. Scroll to Ask about this report.
 2. Type a question related to your report, for example:
 - *"What does my hemoglobin mean?"*
 - *"Is my fasting sugar high?"*
 3. Multilingual Text-to-Speech Support: Integrated an accessibility-focused text-to-speech feature that allows users to listen to AI-generated responses in both English and Urdu, improving usability for users with limited reading proficiency, visual difficulties or in areas with low literacy rate..
-

View health timeline and trends

Navigation: Sidebar → Timeline

1. Open Health Trends.
 2. If you have uploaded at least one report before, a line chart shows how numeric metrics change across report dates.
 3. Use Filter Metric to:
 - Show All Metrics - multiple lines on one chart
 - Select a single test - focus on one biomarker
 4. Hover over the chart to see exact values and dates.
 5. Graph will be plotted connecting all the data points.
-

View report history (audit logs)

Navigation: Sidebar → History

1. Open Audit Logs to see every report processed for your account.
 2. The table shows:
 - Date - when the report was saved
 - AI Summary - short explanation from that session
 - Extracted Raw Data - JSON of test name/value pairs
 3. Use this page to review past analyses without re-uploading files.
-

How the system works (brief overview)

The application uses a layered architecture:

Layer	Technology	Role
Presentation	Next.js (React)	Login, upload UI, charts, chat
Processing	FastAPI + PyMuPDF + Groq AI	PDF Parsing, AI Vision extraction, summaries, Q&A
Data	Supabase (PostgreSQL + Auth)	User accounts and report history

Typical flow: Upload → PDF parsing / AI Vision reading → AI structures metrics → Results saved → Timeline and History read stored data.

Limitations

- Does not provide medical diagnosis or treatment advice.
- Accuracy depends on report quality, OCR clarity, and AI interpretation.

- Processing time may exceed a few seconds for large PDFs or slow networks.